

DigitalDSA

Navigation & UI Glitch Resolution Report

Session 46 | March 31, 2026 | Internal Technical Document

Executive Summary

This report documents four navigation and UI glitch issues identified and resolved in Session 46. Two were **High severity** (500 errors, page stacking) and two were **Medium severity** (visual flash, broken buttons). All issues stemmed from SvelteKit-specific timing and state management patterns. The fixes span 14 files with 139 lines added and 51 removed.

1. Login/Logout Page Stacking **[High]**

Problem

After OTP verification and login, the login page remained visible on screen while the dashboard rendered below it, creating a stacked appearance. The same issue occurred on logout — the dashboard stayed visible with the login page appearing below it. Users had to scroll to see the dashboard.

Root Cause

- All `goto()` calls in `loginWithRole()`, `restoreAccount()`, `startFresh()`, `handleLogout()`, and `handleDeleteAccount()` were **not awaited**.
- The `finally{}` blocks executed immediately after the un-awaited `goto()`, resetting `iswaiting = false` before navigation completed.
- This triggered a re-render of the current page component while SvelteKit was transitioning to the new route, causing both page components to exist in the DOM simultaneously.

Resolution

- Added `await` to all `goto()` calls — 5 in login page, 3 in dashboard layout, 1 each in `DemoBanner` and `DemoRestrictionModal`.
- Removed `finally { iswaiting = false }` blocks. On successful navigation the component is destroyed, so state reset is unnecessary. Moved `iswaiting = false` to `catch{}` only (error recovery path).
- Logout on landing page changed from `goto('/login')` to `window.location.href = '/login'` for reliable hard navigation.

Files Modified

```
src/routes/(auth)/login/+page.svelte
src/routes/dashboard/+layout.svelte
src/lib/components/DemoBanner.svelte
src/lib/components/DemoRestrictionModal.svelte
```

2. Case Intake Page Flash on Form Reload **[Medium]**

Problem

When a user reloads any of the 6 loan form pages (home-loan, LAP, plot-loan, business-loan, personal-loan, professional-loan) mid-progress, the "Case Intake" page (page 0) flashed briefly before the resume modal appeared. The flash was visible long enough to be screenshot-captured. This happened even though the resume

modal itself has a fully opaque overlay.

Root Cause

- **Layer 1 — SSR always renders page 0:** The `+page.server.ts` evaluates `pageIndex=0` with empty answers. `currentPageIndex` initializes to 0 and `showResumeModal` starts false.
- **Layer 2 — onMount timing gap:** The resume modal is only triggered in `onMount()`, which runs AFTER the first render. During SSR and initial hydration, page 0 content is visible without any overlay.
- **Layer 3 — sessionStorage unavailable during SSR:** `formState.currentPageIndex` returns 0 during SSR (no `sessionStorage` on server), so the overlay gate evaluated to "ready" during server rendering.

Resolution

- Created `FormLoadingOverlay.svelte` — fully opaque fixed overlay (`position: fixed; inset: 0; z-index: 9999`) with a spinner. Supports dark mode via `:global(.dark)` selector.
- Added `formReady = $state(false)` to all 6 form pages — **always starts false** so the overlay is present in SSR HTML with zero hydration gap.
- `formReady` transitions to true at 4 code paths: (A) `onMount` when no saved progress, (B) edit mode, (C) restart/clear resume choice, (D) `.finally()` after `evaluateOnServer()` completes on resume.

Files Modified

```
src/lib/components/form-wizard/FormLoadingOverlay.svelte (NEW)
src/routes/(app)/form/home-loan/+page.svelte
src/routes/(app)/form/lap/+page.svelte
src/routes/(app)/form/plot-loan/+page.svelte
src/routes/(app)/form/unsecure-loan/business-loan/+page.svelte
src/routes/(app)/form/unsecure-loan/personal-loan/+page.svelte
src/routes/(app)/form/unsecure-loan/professional-loan/+page.svelte
```

3. 500 Error on /form/how-can-we-help Reload **[High]**

Problem

Reloading the "How Can We Help" page (loan type selection) while authenticated always returned a 500 Internal Server Error. The error page showed no useful details — just "No error details provided" with code UNKNOWN_ERROR. The "Go Home" and "Retry" buttons on the error page had limited functionality.

Root Cause

- `schemaUtils.ts` function `preprocessSchemaBindings()` was trying to set `item.bindsTo = ...` directly on the schema object.
- The `schemaLoader.ts` applies `deepFreeze()` to all cached schemas at startup to prevent accidental mutation.
- This caused `TypeError: Cannot add property bindsTo, object is not extensible` during SSR when the page component called `preprocessSchema()`.

Resolution

- Changed `processItem()` to clone each item with `{ ...item }` spread before mutating properties. The original frozen object is never touched.
- Added try-catch around the `Applicant.findOne()` DB query in `(app)/+layout.server.ts` so a MongoDB failure doesn't 500 the entire form route tree.

Files Modified

```
src/lib/utils/schemaUtils.ts
src/routes/(app)/+layout.server.ts
```

4. Dashboard/Logout Buttons Broken After Error Recovery **[Medium]**

Problem

After encountering a 500 error and clicking "Go Home" to return to the landing page, the Dashboard and Logout buttons in the navigation bar did nothing when clicked. The page appeared normal but these interactive elements were non-functional.

Root Cause

- Both buttons used `goto()` (SvelteKit client-side navigation) which requires a fully hydrated SvelteKit router to function.
- The "Go Home" link on the error page uses `<a href="/">` (standard HTML navigation), which causes a full page load.
- After a full page load triggered by error recovery, SvelteKit's client-side router was not reliably initialized, causing `goto()` calls to fail silently.

Resolution

- Dashboard button changed from `<button onclick={goto()}>` to `<a href={dashboardHref}>` — works with or without JavaScript hydration.
- Logout changed to `window.location.href = '/login'` (hard navigation) with fallback cookie clearing (`document.cookie`) if the logout API fails.
- Removed unused `goto` import from `FloatingNav`.

Files Modified

```
src/lib/components/landing/FloatingNav.svelte
```

Summary

Issue	Severity	Root Cause	Fix Approach
Login/Logout page stacking	High	Un-awaited goto() + finally{} re-render race	await goto(), move state reset to catch{}
Case Intake flash on reload	Medium	SSR renders page 0, overlay only after hydration	formReady=false always, overlay in SSR HTML
500 on how-can-we-help	High	Mutating deep-frozen schema objects	Clone items with spread before mutating
Broken nav after 500 error	Medium	goto() needs hydrated SvelteKit router	Use <a href> and window.location.href

Recommendations for Team

- 1. Audit all goto() calls in try/finally blocks.** Any goto() inside a try block should be awaited. State resets (isLoading = false, etc.) should only happen in catch{}, never in finally{}, because on successful navigation the component is destroyed.
- 2. Never mutate schema objects directly.** All schema preprocessing must clone before modifying. The deepFreeze() in schemaLoader catches this at runtime, but consider adding a lint rule or code review checklist item to prevent future occurrences.
- 3. Prefer <a href> for critical navigation buttons.** Landing page nav buttons (Dashboard, Logout) should use standard HTML links rather than client-side goto(). This ensures they work even when SvelteKit's client router isn't fully initialized.
- 4. SSR-safe loading states must default to "loading".** Any UI gate that depends on browser-only APIs (sessionStorage, localStorage) should default to the blocked/loading state during SSR. Initialize \$state(false), not \$state(browserCheck), and let onMount resolve the correct state.
- 5. Add a handleError hook for better error visibility.** Currently the app has no handleError export in hooks.server.ts. Server-side errors show as "No error details provided". Adding a handleError hook with structured logging would make debugging significantly faster.

Commits: 16bd85aa, 87097129, c4b0c5e6, 38a0aa6f | 14 files changed | +139 / -51 lines | Type check: 0 errors